# Microcontroller Clocks and Interrupts

This guide provides an in-depth look at two core microcontroller (MCU) subsystems: the Clock Tree, which supplies timing to the processor and peripherals, and the Interrupt system, which enables efficient event-driven responses. Together they form the timing "heart" and event-handling "nervous system" of an MCU.

## Part I — Microcontroller Clock Systems

Clocks are the most critical element of an MCU. They provide the timing signals necessary for the processor to execute instructions and for peripherals (Timers, ADCs, GPIOs, etc.) to function.

### 1. The Clock Tree Concept

The Clock Tree is a hierarchical representation of how a clock signal travels from a primary source to various internal components. Understanding it is essential for configuring peripheral timing.

- **Source-to-Peripheral Path:** A clock signal originates at a source, passes through the Reset and Clock Control (RCC) unit, may be multiplied or divided by prescalers, and is finally delivered to the peripheral.
- **Clock Distribution:** A single source can branch out to multiple "branches" of the tree, and each branch can run at its own frequency depending on the connected hardware's requirements.

### 2. Clock Sources

MCUs typically support multiple clock sources, categorized by location and physical implementation:

| Source | Description | Typical Frequency | Characteristics |
|---|---|---|---|
| HSI (High-Speed Internal) | Internal RC oscillator circuit | ~8 MHz | No external components needed; less precise |
| HSE (High-Speed External) | External crystal, ceramic resonator, or RC | 4 MHz – 16 MHz | High precision; requires external pins |
| LSE (Low-Speed External) | External crystal for low-power tasks | ~32.768 kHz | Used for Real-Time Clocks (RTC) |
| PLL (Phase-Locked Loop) | Internal multiplier/divider | Up to max CPU limit | Boosts source frequencies (e.g. 8 MHz → 72 MHz) |

### 3. Reset and Clock Control (RCC)

The RCC is the peripheral responsible for managing the clock tree and power consumption.

- **Default State:** In advanced MCUs like the STM32, clocks to most peripherals are disabled by default to conserve power.
- **Enabling Peripherals:** Before using a peripheral (e.g. GPIO or ADC), the developer must enable its clock via the RCC registers. If the clock is not enabled, the peripheral will not respond to register writes.

- **Comparison:** Unlike the STM32, some older or simpler MCUs (e.g. ATmega32) provide clocks to all modules automatically, without a dedicated RCC management layer.

### 4. Bus Architectures and Frequency Limits

Clock signals are distributed via internal buses. Each bus has a maximum frequency limit that must not be exceeded:

| Bus | Role | Typical Max Frequency |
|---|---|---|
| AHB (Advanced High-performance Bus) | Usually the system bus connected to the CPU | 72 MHz (often the highest in the system) |
| APB1 (Advanced Peripheral Bus 1) | Typically a low-speed peripheral bus | 36 MHz |
| APB2 (Advanced Peripheral Bus 2) | Often a high-speed peripheral bus | 72 MHz |

⚠️ **Warning:** Exceeding these limits through improper PLL or prescaler configuration can cause system instability or hardware damage.

## Part II — Microcontroller Interrupts

Interrupts allow an MCU to respond to asynchronous events immediately, without constantly checking (polling) the status of a peripheral.

### 1. The Interrupt Concept

An interrupt is an event that forces the CPU to pause its current execution, save its state, and execute a specific function called an Interrupt Service Routine (ISR). Once the ISR finishes, the CPU returns to the exact point where it was interrupted.

### 2. Core Components of Interrupt Handling

#### Interrupt Service Routine (ISR)

The ISR is a function that contains the code to be executed when a specific interrupt occurs.

- **Key Rule:** ISRs should not be called manually in the code — they are triggered only by hardware.
- **Variables:** Any global variable modified inside an ISR and used elsewhere in the code should be declared with the volatile keyword to prevent compiler optimization errors.

#### Interrupt Vector Table (IVT)

- The IVT is a table in memory (usually at the start of Flash) that stores the addresses of all ISRs.
- Each interrupt has a unique ID or index in this table.
- When an interrupt occurs, the hardware looks up the corresponding address in the IVT to know where to "jump" to execute the ISR.

### 3. Interrupt Controller Architecture

Modern MCUs use a dedicated hardware block to manage interrupts, often referred to as the GIC (General Interrupt Controller) or NVIC (Nested Vectored Interrupt Controller).

- **Role of the Controller:** It acts as a middleman between the peripherals and the CPU: it prioritizes multiple simultaneous interrupts and signals the CPU via a single "Interrupt Request" (IRQ) line.

Handshaking sequence:

- **Request:** The controller sends a signal to the CPU.
- **ID Transfer:** The controller provides the ID of the specific interrupt.
- **Acknowledgment:** The CPU responds with an acknowledgment signal when it begins handling the interrupt.

### 4. Interrupt Types

- **Hardware Interrupts:** Triggered by physical peripherals (e.g. ADC conversion complete, Timer overflow, or a button press).
- **Software Interrupts:** Triggered by specific assembly instructions (e.g. SWI) to switch execution modes or handle system calls.

## Part III — Signal Routing and External Interrupts (EXTI)

### 1. Pin Multiplexing and Alternate Functions

To reduce the physical size and cost of the MCU, a single physical pin is often shared by multiple internal peripherals. This is managed by the AFIO (Alternate Function I/O) or similar routing logic.

- **Multirouting:** A pin can be configured as a standard GPIO, a UART transmission line, or an external interrupt trigger.
- **Configuration:** The developer must select the "Alternate Function" in the configuration registers to route the physical pin to the desired internal peripheral.

### 2. The EXTI (External Interrupt) Path

When a signal from an external pin needs to trigger an interrupt, it follows a specific path:

| Step | Stage | Description |
|---|---|---|
| 1 | External Pin | A signal (e.g. a falling edge from a button) arrives. |
| 2 | AFIO / Routing | The pin is routed to the EXTI controller. |
| 3 | EXTI Controller | Detects the specific trigger (rising edge, falling edge, or level) and generates an interrupt request. |
| 4 | Interrupt Controller (GIC/NVIC) | The EXTI request is prioritized and sent to the CPU. |
| 5 | CPU | Pauses execution and jumps to the ISR address found in the IVT. |

# Part IV — Practical Configuration Workflow

When setting up a project, the following logical sequence is typically followed:

## Step 1 — Clock Setup

- Identify the desired System Clock frequency.
- Configure the Clock Source (HSI/HSE).
- Adjust PLL and prescalers via the RCC to ensure buses (AHB/APB) and peripherals stay within their frequency limits.
- Enable the clock for every peripheral intended for use.

## Step 2 — Interrupt Setup

- Configure the peripheral to generate an interrupt (e.g. set the "End of Conversion" interrupt bit in an ADC).
- Configure the Interrupt Controller (NVIC/GIC) to enable that specific Interrupt ID.
- Define the ISR function in the code and ensure the IVT is properly initialized (usually handled by startup code).
- If using external pins, configure the AFIO/EXTI path to route the pin signal to the interrupt controller.